

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет прикладної математики та інформатики
Кафедра математичного моделювання соціально-економічних процесів

Звіт

Лабораторна робота №1

Виконав:

студент групи Пма-42

Думанський Юрій

Перевірила:

доцент Лисецька О. Ю.

Львів-2026

Тема:

Робота з різними типами даних. Анотування

Мета:

Опанувати механізми статичної типізації в Python, навчитися застосовувати рекурсивні анотації та створювати узагальнені класи (generics) для підвищення надійності коду.

Завдання:

1. Реалізувати клас `TreeNode` для представлення бінарного дерева з використанням анотацій типів та модуля `future`.
2. Реалізувати узагальнений клас `ResourceManager` для керування ресурсами довільного типу з використанням `TypeVar` та `Generic`.

Хід роботи:

- Для реалізації рекурсивного посилання на клас `TreeNode` всередині його власного визначення було використано імпорт `'from future import annotations'`. Це дозволило уникнути помилок визначення типів на етапі ініціалізації об'єктів. Атрибути `left` та `right` типізовано як `Optional[TreeNode]` для коректної обробки порожніх вузлів.
- При розробці класу `ResourceManager` використано механізми метапрограмування Python: оголошено `TypeVar('T')` для представлення довільного типу даних. Клас успадковано від `Generic[T]`, що забезпечило можливість перевірки відповідності типів при оновленні та отриманні ресурсів. Додатково впроваджено декоратор `@property` для альтернативного доступу до даних.
- Тестування проводилося на наборах даних типів `int`, `str` та `list`. Для верифікації анотацій використано статичний аналізатор `mypy` з прапорцем `--strict`. Під час аналізу було забезпечено повну відповідність типів повернутих значень та аргументів методів, що підтверджено відсутністю помилок у звіті аналізатора.

Весь код наведений в додатку 1.

Результати:

Клас `TreeNode` успішно формує деревоподібну структуру та коректно визначає листки (вузли без нащадків).

```
--- Тести TreeNode ---
left_leaf is leaf? True (Очікується: True)
root is leaf? False (Очікується: False)
partial_node is leaf? False (Очікується: False)
-----
```

Клас `ResourceManager` забезпечує роботу з різними типами даних, автоматично підхоплюючи тип при ініціалізації об'єкта.

```
--- Тести ResourceManager ---
Тип ресурсу: int
Оновлений int ресурс: 200
Тип ресурсу: str
Оновлений str ресурс: Оновлений рядок через property
Тип ресурсу: list
Поточний list ресурс: [1, 2, 3]
-----
```

Весь код пройшов повну статичну перевірку модулем `муру`:

```
Success: no issues found in 2 source files
```

Висновки:

У ході роботи вивчено інструменти типізації Python, зокрема модулі `typing` та `future`. Практично доведено, що використання `Generic` та рекурсивних анотацій дозволяє створювати гнучкі та водночас захищені від помилок структури даних. Використання `муру` на етапі розробки дозволяє виявляти помилки невідповідності типів до моменту виконання коду.

Додаток 1. Код програми.

Частина 1.

```
from __future__ import annotations
from typing import Optional

class TreeNode:
    def __init__(self, value: int, left: Optional[TreeNode] = None, right:
Optional[TreeNode] = None) -> None:
        self.value: int = value
        self.left: Optional[TreeNode] = left
        self.right: Optional[TreeNode] = right

    def is_leaf(self) -> bool:
        """Повертає True, якщо вузол не має нащадків (є листком)."""
        return self.left is None and self.right is None

def test_tree_node() -> None:
    left_leaf = TreeNode(value=10)
    right_leaf = TreeNode(value=20)

    root = TreeNode(value=15, left=left_leaf, right=right_leaf)

    partial_node = TreeNode(value=5, left=TreeNode(2))

    print("--- Тести TreeNode ---")
    print(f"left_leaf is leaf? {left_leaf.is_leaf()} (Очікується: True)")
    print(f"root is leaf? {root.is_leaf()} (Очікується: False)")
    print(f"partial_node is leaf? {partial_node.is_leaf()} (Очікується: False)")
    print("-" * 20)

if __name__ == "__main__":
    test_tree_node()
```

Частина 2.

```
from __future__ import annotations
from typing import TypeVar, Generic

T = TypeVar('T')

class ResourceManager(Generic[T]):
    def __init__(self, resource: T) -> None:
        self._resource: T = resource

    def update_resource(self, new_resource: T) -> None:
        """Оновлює значення ресурсу"""
        self._resource = new_resource

    def get_resource(self) -> T:
        """Повертає поточний ресурс"""
        return self._resource

    def log_type(self) -> None:
        """Виводить тип ресурсу у консоль."""
        print(f"Тип ресурсу: {type(self._resource).__name__}")

    @property
    def resource(self) -> T:
        return self._resource

    @resource.setter
```

```

def resource(self, new_resource: T) -> None:
    self._resource = new_resource

def test_resource_manager() -> None:
    print("--- Тести ResourceManager ---")

    # Тест 1: Цілочисельний ресурс (int)
    int_manager = ResourceManager[int](100)
    int_manager.log_type()
    int_manager.update_resource(200)
    print(f"Оновлений int ресурс: {int_manager.get_resource()}")

    # Тест 2: Рядковий ресурс (str) з використанням @property
    str_manager = ResourceManager[str]("Початковий рядок")
    str_manager.log_type()
    str_manager.resource = "Оновлений рядок через property"
    print(f"Оновлений str ресурс: {str_manager.resource}")

    # Тест 3: Списковий ресурс (list)
    list_manager = ResourceManager[list[int]]([1, 2, 3])
    list_manager.log_type()
    print(f"Поточний list ресурс: {list_manager.get_resource()}")
    print("-" * 20)

if __name__ == "__main__":
    test_resource_manager()

```